PROJECT REPORT

Trekking Management Application

Modern Application Development - 2

Vaishnavi Saxena

Roll No. - 23f2003009
Email: 23f2003009@ds.study.iitm.ac.in

PROBLEM STATEMENT

The project aims to develop a platform that facilitates interaction between Users (Trekkers) and Trekking staff — enabling trekkers to browse and book treks, trek staff to manage the treks assigned to them and view participant details, all while under the administration of a superuser (Admin).

APPROACH

The app was built using Flask as the backend framework, with SQLAlchemy for ORM-based data modeling and Flask-JWT-Extended for stateless, token-based authentication. The frontend was built using Vue (via Vite) with Vue Router for client-side navigation and Bootstrap 5 for styling. The application supports three distinct user roles — Admin, Trek Staff, and Trekker — each with its own dedicated dashboard, sidebar navigation, and permitted set of actions, enforced independently on both frontend (for UX) and backend (for actual security). A key thing to mention is that throughout the application we use **soft-deletes** instead of hard-deletes – eg: *closing* a trek, *deactivating* a user and *cancelling* a booking instead of deleting them from the DB.

AI/LLM DECLARATION

I used Claude (Anthropic) to assist with generating Flask route boilerplate, Vue component structure and for debugging error messages, while all architectural decisions, DB schema designing, business logic choices and final integration/testing were done by me.

FRAMEWORKS USED

1. **Bootstrap** — for frontend styling
2. **Flask** — backend
3. **Flask SQLAlchemy** — ORM for SQLite database
4. **Flask JWT Extended** — Token based authentication
5. **Flask CORS** — for cross port communication between frontend and backend
6. **Flask Caching and Redis** — server side response caching
7. **SQLite** — relational DB for data storage
8. **Vue CLI (Vite)** — frontend
9. **Axios** — HTTP client for frontend-backend API communication
10. **Werkzeug** — password hashing and verification

API ENDPOINTS

Endpoint	Method	Description
`/api/register`	POST	Register a new trekker account
`/api/login`	POST	Authenticate user and issue JWT access token
`/api/treks`	GET	Fetch all open treks (cached)
`/api/treks/<id>`	GET	Fetch details for a single trek
`/api/bookings`	POST	Create a new booking for the logged-in trekker
`/api/my-bookings`	GET	Fetch the logged-in trekker's active bookings
`/api/trekker/dashboard`	GET	Fetch trekker's dashboard summary data
`/api/trekker/history`	GET	Fetch trekker's past (cancelled/completed/invalidated) bookings
`/api/trekker/profile`	GET / PUT	View or update the trekker's own profile
`/api/trekker/profile/password`	PUT	Change the trekker's own password
`/api/staff/dashboard`	GET	Fetch staff member's dashboard KPIs and assigned treks

`/api/staff/my-treks`	GET	Fetch treks assigned to the logged-in staff member
`/api/staff/treks/<id>/participants`	GET	Fetch participant list for a specific assigned trek
`/api/staff/treks/<id>/close`	PUT	Close a trek assigned to the staff member
`/api/staff/participants`	GET	Fetch all participants across the staff member's assigned treks
`/api/staff/profile`	GET / PUT	View or update the staff member's own profile
`/api/staff/profile/password`	PUT	Change the staff member's own password
`/api/admin/dashboard`	GET	Fetch admin dashboard overview KPIs
`/api/admin/treks`	GET / POST	List all treks (with filters) / create a new trek
`/api/admin/treks/<id>`	PUT	Update an existing trek
`/api/admin/treks/<id>/close`	PUT	Close trek (soft-delete, cascade: invalidate bookings)
`/api/admin/treks/<id>/reopen`	PUT	Reopen a previously closed trek
`/api/admin/staff`	GET / POST	List active staff / create a new staff account
`/api/admin/users`	GET	List trekkers (with active/inactive filter)
`/api/admin/users/<id>/status`	PUT	Activate or deactivate any user
`/api/admin/bookings`	GET	List all bookings (with status filter)
`/api/admin/bookings/<id>/cancel`	PUT	Cancel a specific booking (restores the trek slot)
`/api/admin/search`	GET	Search across treks, staff, trekkers, or bookings
`/api/admin/settings`	GET / PUT	View or update the admin's own profile
`/api/admin/settings/password`	PUT	Change the admin's own password

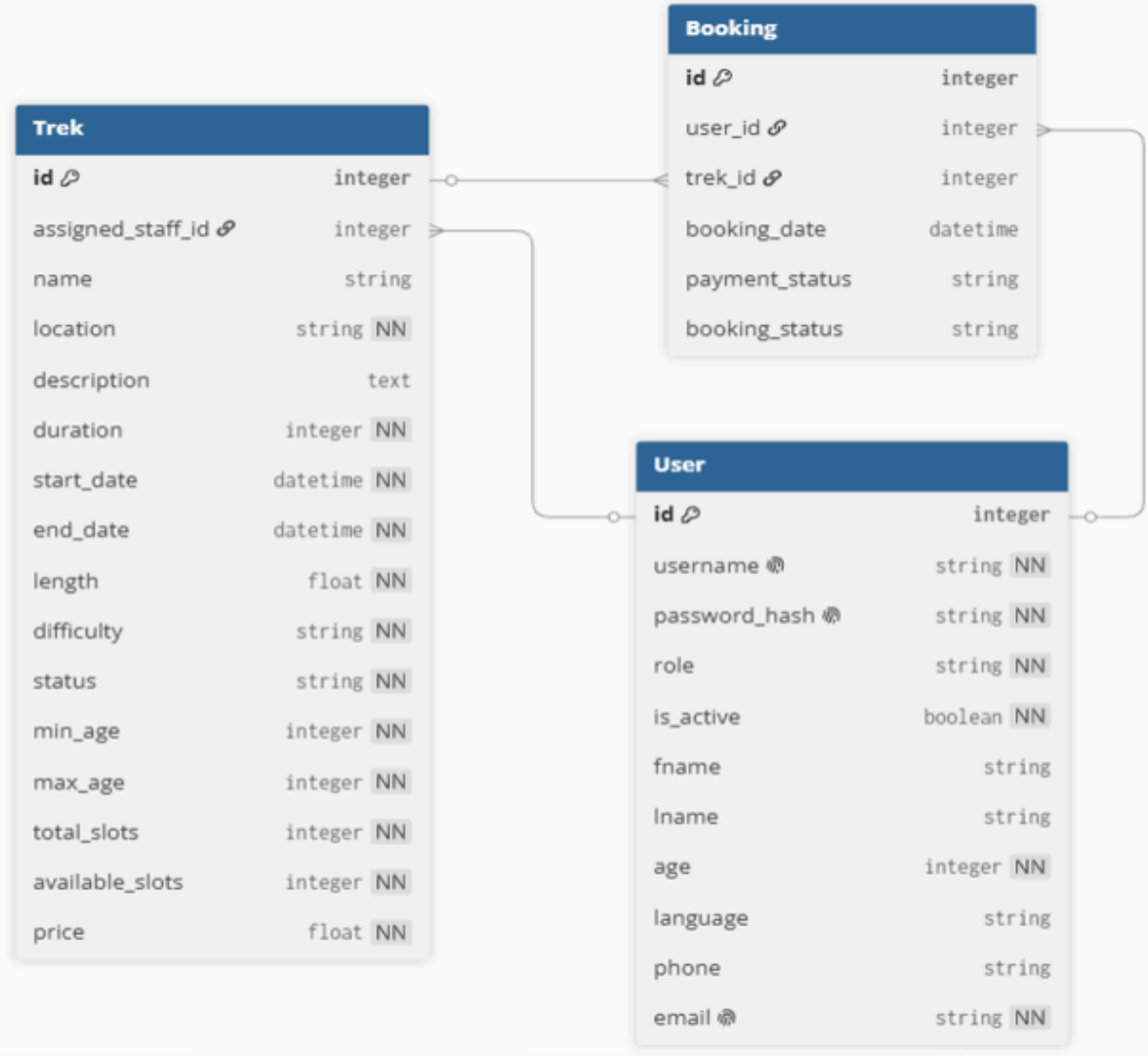
ENTITY-RELATIONSHIP DIAGRAM

TABLES

- USER - stores user (trekker, staff and admin) profile details (attribute as shown in the schema below)
- TREK - stored a particular trek route details. Treks are created by superuser (Admin) and assigned to a staff member.
- BOOKING - stores booking records. Bookings are made by users against treks

RELATIONSHIPS

- **User** → **Booking** — *one to many*. A trekker can have multiple bookings
- **User** → **Trek** — *one to many.* A staff member can be assigned to multiple treks
- **Trek** → **Booking** — *one to many.* A trek can have multiple bookings



FOLDER ARCHITECTURE

'TMA_MAD2_Project' is the project's root folder. Within the root folder, we have —

1. A folder **'backend'** that contains —
    - '**app.py**': main Flask app entry point with all API routes
    - **'models.py'** : with the schema/model definition of the database.
2. A folder **'instance'** that contains —
    - **'trekking.db'** : the database file.
3. A folder **' frontend'** that contains —
    - A folder **'src/views'** : page level Vue components organized by role (trekker/, staff/, admin/)
    - **'/src/components'** : reusable Vue components eg: nav-bars and modal
    - **'src/router/index.js'**: route definitions with role-based navigation guards

PROJECT FEATURES

The flask app opens with the welcome page that navigates the client to the login/registration page. The functionalities the app provides are:

- User registration (trekkers only) and role-aware login (admin/staff/trekker) with JWT authentication
- Role-based sidebar layouts and route guarding
- **Trekker:** browse open treks, book via a modal, view active bookings, view booking history and edit profile
- **Trek Staff:** view/close assigned treks, view participants via modal, edit profile
- **Admin:** full CRUD for treks, staff, trekkers and bookings; cross entity search.
- Server side response caching for one API – GET `api/trekker/dashboard.`

PRESENTATION VIDEO LINK

TMADemo_23f2003009.mp4